

1:n Assoziation

Merkmal	Beschreibung
Definition	Eine 1:n Assoziation beschreibt eine Beziehung zwischen zwei Klassen, bei der eine Instanz der einen Klasse mit mehreren Instanzen der anderen Klasse verbunden ist.
Unidirektionale Assoziation	Eine Klasse besitzt eine Collection von Referenzen auf Objekte der anderen Klasse, aber nicht umgekehrt. Beispiel: Eine <code>Library</code> verwaltet mehrere <code>Books</code> , aber ein <code>Book</code> kennt die <code>Library</code> nicht direkt.
Bidirektionale Assoziation	Beide Klassen besitzen Referenzen aufeinander. Beispiel: Ein <code>Customer</code> kennt seine <code>orders</code> , und jede <code>order</code> speichert den zugehörigen <code>Customer</code> .
Datenstruktur	Die N-Objekte werden typischerweise in einer <code>List<T></code> , <code>Set<T></code> oder <code>Map<K, V></code> gespeichert.
Aggregation vs. Komposition	Eine 1:n Assoziation kann entweder eine Aggregation (schwache Abhängigkeit) oder eine Komposition (starke Abhängigkeit, existenzgebunden) sein.
Referenzverwaltung	Die Hauptklasse verwaltet die Objekte der Unterklasse und stellt Methoden zur Hinzufügung, Entfernung und Abfrage bereit.
Lebenszyklus	Bei einer Komposition existieren die N-Objekte nur, solange das 1-Objekt existiert. Bei einer Aggregation können sie unabhängig existieren.

Zentrale Methoden

Methode	Beschreibung
<code>add(T element)</code>	Fügt ein Element zur <code>collection</code> hinzu.
<code>remove(T element)</code>	Entfernt ein Element aus der <code>collection</code> .
<code>getAll()</code>	Gibt alle zugeordneten Objekte zurück.
<code>size()</code>	Gibt die Anzahl der zugeordneten Elemente zurück.
<code>clear()</code>	Löscht alle zugeordneten Objekte.
<code>contains(T element)</code>	Prüft, ob ein Element in der <code>collection</code> existiert.
Bidirektionale Synchronisation	In einer bidirektionalen Beziehung sorgt eine Methode dafür, dass sich beide Objekte gegenseitig referenzieren (z. B. <code>order.setCustomer(this)</code>).

Anwendungsszenarien

Szenario	Beschreibung
Customer und Orders	Ein Customer kann mehrere Orders haben, aber jede Order gehört genau zu einem Customer.
Library und Books	Eine Library besitzt mehrere Books, aber jedes Book ist nur in einer Library vorhanden.
Company und Employees	Eine Company beschäftigt mehrere Employees, aber jeder Employee ist genau einer Company zugeordnet.
School und Students	Eine School verwaltet viele Students, aber jeder Student gehört genau zu einer School.
Author und Books	Ein Author kann mehrere Books schreiben, aber jedes Book hat nur einen Author.
Country und Cities	Ein Country besitzt mehrere Cities, aber jede City gehört nur zu einem Country.